ECE496 Implementation Plan
Team 2025251: DeFi AI Agent

Supervisor: Andreas Veneris
Administrator: Eric Lefebvre **Section:** 5
Students: Ozod Musaev, Sami Najim, Youssef Fikry, George Li
Project ID: 2025251
Date of Submission: November 8, 2025

1.0 Summary of Progress Status and Changes

This section provides a structured overview of our project's current status, challenges, and discusses updates that we have made since the project proposal.

1.1 Implementation Progress

- Learning and Understanding DeFi and ElizaOS: The team conducted an in depth exploration of decentralized finance concepts and the ElizaOS framework, providing us with a strong understanding of the tools that will be used for our AI Chatbot.
- Replication of Spartan AI Trading Bot: We have successfully forked and run Spartan locally, which will act as a baseline execution engine for our project
- Implementation of Core Components (In progress)
    - Chat & UX: we are currently working on integrating the UX using Telegram as the interface
    - NLU/Intent Parser: Our current version of the parser using simple regex and simple grammar is functional and converts natural-language commands into a structured JSON format

1.2 Challenges Encountered

- Spartan AI Trading Bot Replication Issues: While testing the Spartan AI locally, we ran into several conflict dependencies related to version mismatches and package updates. Connecting Spartan to the testnet environment also required reconfiguring RPC endpoints and updating API keys, which we were previously unfamiliar with.
- Natural Language Parsing Edge Cases: While our initial NLU parser performs well for basic commands, it struggles with ambiguous and nested actions. We are currently considering LLM-assisted fallback and clarification for better error handling for ambiguous inputs.

1.3 Plans Ahead

- Module Integration: Complete the integration NLU & Intent Parser, Router & Quote Engine, and Spartan→EVM Interop Layer to enable end-to-end transaction execution as outlined in Section III.
- Simulation and Performance Verification: Implement the Simulation & Risk module on Sepolia to validate transaction safety and measure latency against the SyRS performance targets (≤ 12 s median confirmation).

1.4 Changes Since Proposal

- Refinement of Simulation & Risk Module: The simulation functionality proposed earlier has been expanded Simulation and Risk module that performs pre-execution validation on Sepolia before any trade is broadcasted.

2.0 Possible Solutions and Design Alternatives

This section explores three alternative design approaches considered by the team and presents a justification for the selected solution. Each approach is evaluated based on feasibility, scalability, accessibility, and alignment with the project's SyRS**.**

Alternative 1: Agent-Based Architecture Using Spartan Fork (Selected Approach)

The selected design builds upon an open-source agent execution framework, Spartan, which provides pre-existing decentralized finance integration modules. The team extends this base to implement an AI agent capable of interpreting user text commands and executing transactions.

Advantages

- Combines modularity, reusability, and decentralization.
- Accelerates implementation by leveraging Spartan's existing execution infrastructure.
- Maintains user safety through pre-execution simulation, confirmation prompts, and restricted command sets.
- Aligns directly with the SyRS requirements of accessibility, reliability, and controlled execution.

Disadvantages

- Requires initial effort to understand and extend the Spartan framework.
- Limited documentation for some internal modules may slow early development.

Evaluation

This approach provides the best balance between implementation feasibility and project objectives. By extending Spartan, the team can focus on innovating in AI-driven intent parsing, transaction validation, and natural-language interactivity, rather than rebuilding blockchain infrastructure. The resulting system preserves decentralization while enabling rapid and safe execution.

Alternative 2: Direct Smart Contract Integration

In this approach, the system would directly interact with blockchain networks (e.g., Ethereum Sepolia) via custom-built smart contracts and an in-house backend API. The large language model would parse the user's text command, and the backend would assemble the corresponding smart contract function call (e.g., a Uniswap swap) using Web3 libraries.

Advantages

- Complete control over the transaction execution pipeline and smart contract logic.
- High flexibility in implementing customized security checks or logging features.
- Independence from third-party frameworks or APIs.

Disadvantages

- Requires significant smart contract development, auditing, and ongoing maintenance.
- Increases risk of vulnerabilities or incompatibilities with future DeFi protocol updates.
- Extends development time and complexity.

Evaluation

While this approach maximizes flexibility and transparency, it demands deep blockchain engineering expertise and extensive testing for security. The complexity of managing smart contracts outweighs its benefits for a university capstone timeline.

Alternative 3: API-Based Execution Layer

This design relies on third-party APIs (e.g., Moralis) to handle blockchain transactions. The AI chatbot sends structured requests to the API, which executes trades on behalf of the user. The system focuses mainly on natural language parsing and transaction orchestration rather than on-chain logic.

Advantages

- Simplifies blockchain integration and reduces on-chain programming effort.
- Provides access to real-time data (token prices, gas fees, liquidity pools) through reliable APIs.
- Faster development and easier debugging.

Disadvantages

- Limited transparency and control over low-level execution.
- Dependence on third-party services may introduce costs, rate limits, or outages.

Evaluation

This solution enables faster prototyping but compromises decentralization and user control, two core principles of the project. It is suitable for proof-of-concept systems but misaligned with the long-term goal of creating an open, trust-minimized agent architecture.

Justification for the Selected Design

The Spartan-based agent architecture was chosen because it ensures a robust, scalable, and secure foundation while meeting all core SyRS objectives:

1) Accessibility: enables simple natural-language interaction for users with no blockchain background.

2) Reliability: inherits proven decentralized execution mechanisms from Spartan.

3) Extensibility: allows integration with future protocols through modular adapters.

4) Feasibility: balances development scope with achievable milestones for the capstone timeline

3.0 Technical Design and Implementation

System-Level Overview

We are forking Spartan for execution scaffolding, then migrating its Solana-leaning assumptions to Ethereum (Sepolia first) and exposing the pipeline through a Telegram bot before adding a full GUI. The user flow is deliberately gated:

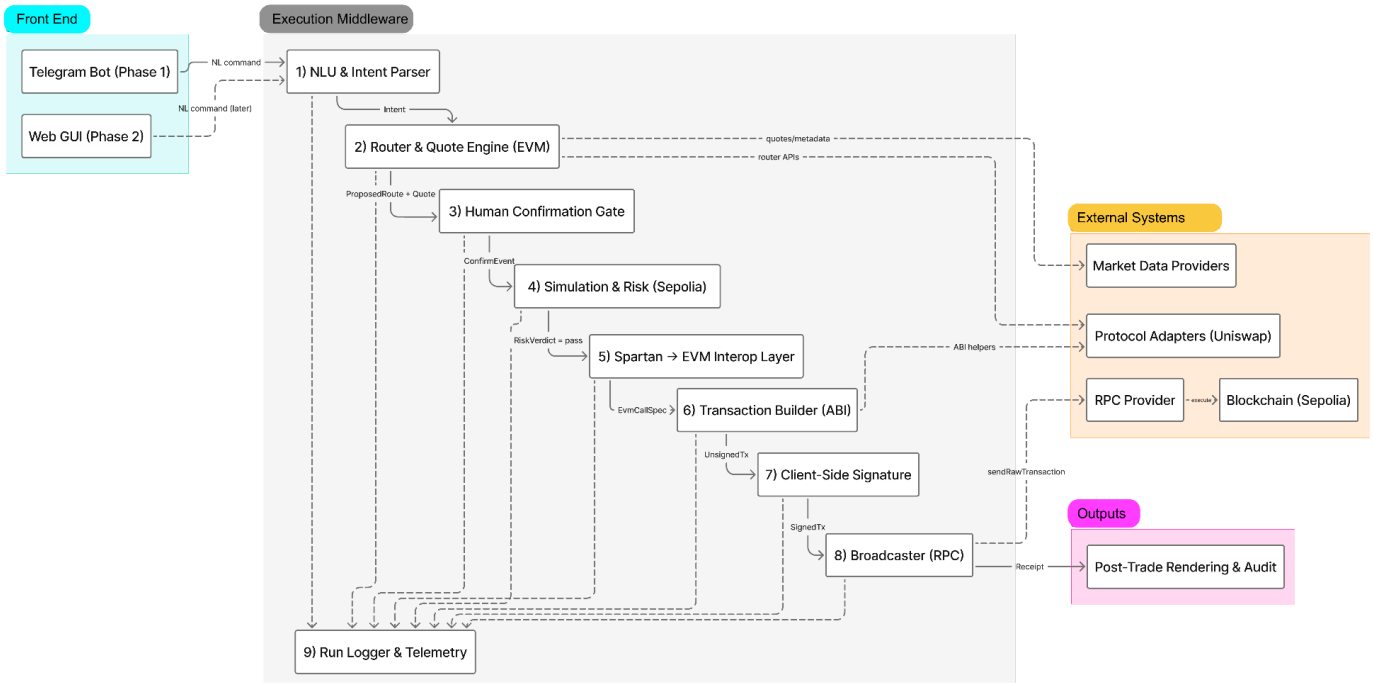
1. Parse
2. Route/quote
3. Confirm
4. Simulate/risk-check
5. Build
6. Sign
7. Broadcast
8. Audit

This aligns with our SyRS: client-side signing (no custody), an explicit Human Confirmation Gate, pre-send simulation with slippage/gas ceilings, Sepolia execution, and per-run pipeline logging with actionable errors.

Spartan's mental model is closer to Solana program instructions; Ethereum wants EVM calldata, nonces, and gas markets. The hard part is a clean Spartan to EVM interop layer that preserves the nice developer ergonomics while emitting standards-compliant Uniswap v2/v3 router calls and meeting our latency and execution-quality objectives on Sepolia.

We sequence deliverables for proof and safety: Telegram E2E first (same backend, minimal UI), then GUI with richer telemetry and revoke/approval tools. This matches the guideline's request for a system-level design that builds from proven blocks and cites prior work we leverage.

3.1 Enhanced System Block Diagram



3.2 Module Descriptions

	Role	I/O	Notes
NLU & Intent Parser	Convert Telegram text into a strict schema.	***In***: NL text ***Out***: Intent	Constrained grammar. Reject partials. Normalize tokens/decimals early.
Router & Quote Engine (EVM)	Quote Uniswap v2/v3. Compute expected out, price impact, and gas.	***In***: Intent, on-chain metadata ***Out***: ProposedRoute, Quote, GasEstimate.	Quote freshness vs. user confirm and slippage tolerance UX. Supports the latency/execution objectives.
Human Confirmation Gate	Block downstream until the user approves a human-readable summary (min-received, max fees, slippage).	***In***: ProposedRoute, Quote ***Out***: ConfirmEvent or Abort.	Enforces “no autonomous trading,” creates audit evidence.
Simulation & Risk	Sepolia dry-run. Enforce balances/allowance, gas ceiling, slippage.	***In***: ConfirmEvent, ProposedRoute ***Out***: SimResult, RiskVerdict.	Precise, actionable errors (e.g., allowance missing vs. slippage breach) so users can self-fix.
Spartan→EVM Interop Layer	Translate Spartan’s Solana-style instructions into EVM-valid calldata	***In***: RiskVerdict=pass, ProposedRoute ***Out***: EvmCallSpec.	• Instruction to ABI mapping to Uniswap router selectors + encoded params.

	with correct nonce and gas policy.		• Token semantics: decimals, ERC-20 approvals, spender addresses vs. Solana accounts. • State/ordering: robust nonce management and idempotent retries (no double-spend). • Gas policy: maxFeePerGas and priority fee under user ceilings. This work unlocks Req-3 and operationalizes the Spartan fork premise.
Transaction Builder (ABI)	Encode calldata; set chainId, nonce, maxFeePerGas; preflight checks.	***In***: EvmCallSpec ***Out***: UnsignedTx.	Schema validation. Min-out enforcement. Parity with simulator.
Client-Side Signature	User signs locally (wallet/test account).	***In***: UnsignedTx ***Out***: SignedTx.	Keys never cross to us. Constraint-9 (no custody).
Broadcaster (RPC)	Submit, poll, and return txHash/receipt.	***In***: SignedTx ***Out***: Receipt.	Verifies the ≤ 12s median / ≤ 25s confirmation target.
Post-Trade Rendering & Audit	Show fills, realized slippage vs. tolerance, gas used. Persist Run ID with pipeline states.	***In***: Receipt, logs ***Out***: user report + audit trail.	Evidence for logging and measurability requirements.

3.3 Cross-Cutting Concerns

Security & Validation

- **Layered safety checks.** We validate each step in order: input format (schema) → sanity-check the quote → run simulation checks → verify ABI/call-data → confirm the signature is valid → confirm the chain receipt is successful.
- **Rate limits & replay protection.** We throttle Telegram commands to prevent spam, track nonces strictly to avoid double-spends, and require a fresh confirmation for any retry.
- **Allowance UX.** If an ERC-20 approval is needed, we ask explicitly, explain the risk, and give a clear place to revoke approvals later.
- **No custody, ever.** Your keys never touch our systems; all signing happens locally on your device/wallet.

Data/Telemetry

- **Market data adapters** cached with short TTL so quotes at confirm are still representative.

- **Run-logger** captures state transitions and timestamps to demonstrate both latency and execution quality at verification time.

Deliverable sequence

- **M1 – Spartan Fork + Interop Skeleton**
- **M2 – Telegram E2E**
- **M3 – Routing & Performance**
- **M4 – GUI + Risk Hardening**

4.0 Test and Verification

This section describes how we will validate that the Ethereum-adapted Spartan pipeline that was outlined in the previous section satisfies the SyRS items such as client-side signing, mandatory human confirmation, pre-send simulation, Sepolia execution, and run-level telemetry. We will have a modular building process of each section separately and then integrate them together.

1. **Module / unit tests**: Each block in Section III will be individually tested with controlled inputs before integration.
2. **Pipeline / integration tests**: Once integrated, the full pipeline "user says swap X→Y" runs on Sepolia, through Telegram user interface.

Test Methods and Setups:

- **Environment:** The environment that we will be using to test includes:
    - Sepolia testnet. Sepolia is a blockchain network used for all on-chain testing without using real Ethereum.
    - Uniswap v2/v3 routers (on Sepolia) - used to simulate swapping or getting quotes with real deployed smart contracts that are on the mainnet.
    - Test ERC-20s - dummy tokens that are used to perform swaps and transfers.
    - RPC provider - used to read data and information from the Sepolia testnet.
    - Telegram bot sandbox - this will be to test the user input/output chatbot interface.
    - Local signer/test wallet - a wallet that holds the test ETH and signs transactions client-side.
- **Pass/Fail style:** Each requirement has an explicit observable that will either meet the expectation or not.

Table 1: Verification Table

ID	System Requirement	How It Will Be Verified
1	The system must understand user text commands (e.g., "swap 0.1 ETH to USDC") and turn them into a structured intent with fields like action, asset, and amount.	**Test:** Send 20+ example messages (valid and invalid ones). Pass if at least 95% are understood correctly, and invalid ones return clear error messages.

2	The system must show a human-readable summary and get user confirmation before any blockchain action.	**Test:** Try to trigger a transaction without confirming. Pass if the system stops and displays “confirmation required.”
3	The system must create valid Ethereum transaction data for a Uniswap v2/v3 swap.	**Test:** Run a swap on the Sepolia test network and compare the transaction data to the expected Uniswap format. Pass if the transaction is accepted.
4	The system must run a “practice” simulation of each trade and block execution if it fails or slippage is too high.	**Test:** Simulate three cases—(a) normal trade, (b) not enough funds, (c) price moved too much. Pass if errors are correctly detected and explained.
5	Confirmed transactions must execute on the Sepolia test network and return a success receipt.	**Test:** Send several test transactions. Pass if all return valid transaction hashes and success receipts.
6	The system must estimate gas/fees and stop if the cost exceeds the user’s limit.	**Test:** Set a very low gas limit and attempt a trade. Pass if the system cancels and warns that the fee is too high.
7	The system must record every stage of the pipeline (parse → route → simulate → sign → send) and never store private keys.	**Test:** After each run, check logs. Pass if all pipeline steps appear with timestamps and no private keys are saved.
8	The system must handle common failures (network error, no funds, bad input) with clear, actionable messages.	**Test:** Intentionally cause these failures. Pass if each message clearly tells the user what went wrong and how to fix it.
9	All signing must happen on the user’s device or a test wallet—never on the server.	**Review + Test:** Check code for key uploads. Run without a local signer. Pass if the system refuses to proceed and keeps the key private.
10	The agent must never trade automatically; only when the user gives a command.	**Test:** Leave the bot idle with no user input. Pass if no transactions are created automatically.
11	The system must reuse Spartan code where possible for execution adapters.	**Review:** Confirm through code inspection that the forked Spartan modules are integrated and referenced.

12	The system's average confirmation time must be fast—median ≤ 12 s, 95% ≤ 25 s.	**Test:** Send 20 trades on Sepolia, record times from "confirm" to "mined." Pass if times meet targets.
13	Successful trades must stay within the user's allowed price difference (slippage).	**Test:** Run 20 swaps with different slippage settings. Pass if at least 95% finish within the user's set tolerance.

5.0 Appendices

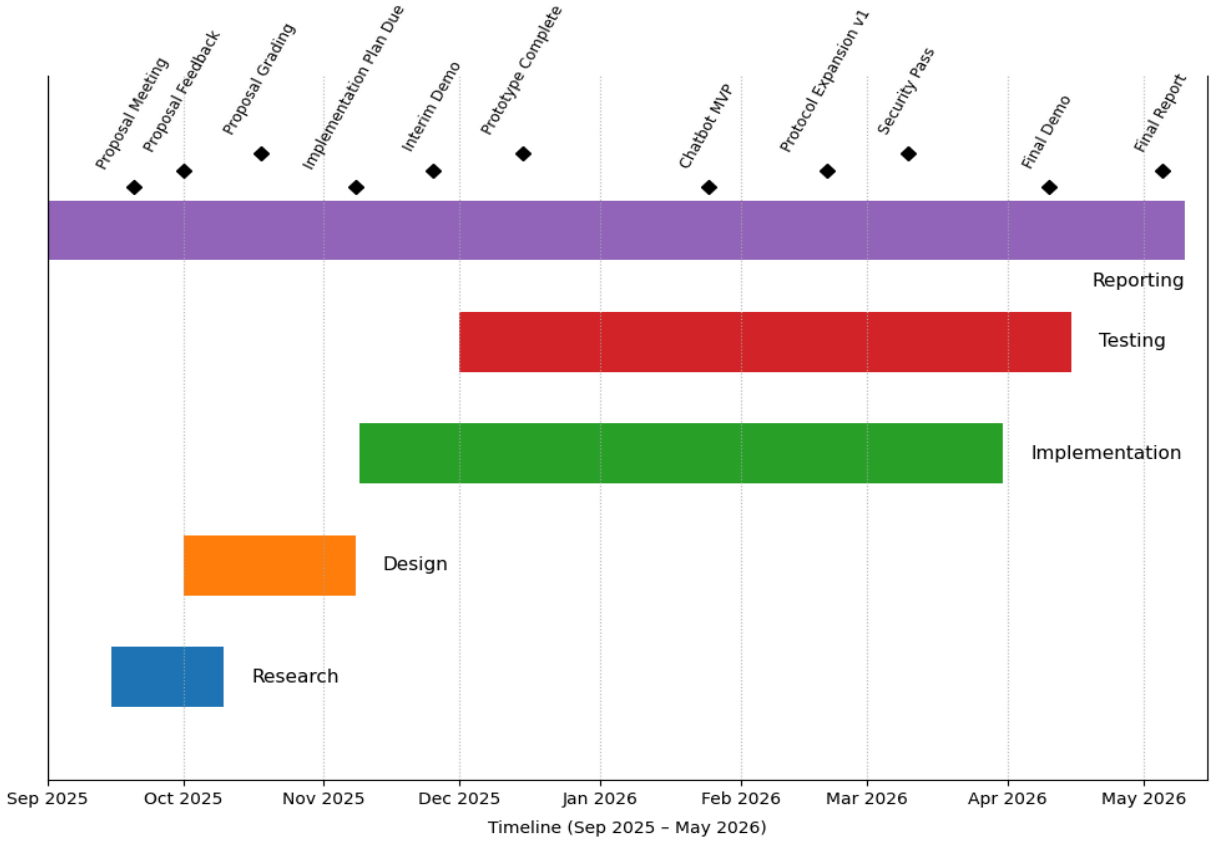
Appendix A: System Requirements Specifications (SyRS)
The following top-level requirements define what the system must achieve to be considered successful. They are organized as Requirements (core functions under our control), Constraints (external limits), and Objectives (graded targets).

Table 2: Requirements Specification Table

ID	Specification	Description
1	The system shall parse a natural-language trade/transfer command into a structured intent with fields (action, asset, amount, chain, tolerance).	**Requirement**: core capability under team control
2	The system shall present a human-readable summary and capture explicit user confirmation before any on-chain action; no transaction is broadcast without a recorded Confirm event.	**Requirement**: verifiable via UI event logs
3	The system shall generate standards-compliant Ethereum transaction call data for at least one DEX swap route (Uniswap v2 or v3 router).	**Requirement**: checked by ABI/static validation + dry-run
4	The system shall simulate each transaction pre-send and shall block execution if simulation fails or estimated slippage exceeds a user-set threshold.	**Requirement**: pass/fail via simulation harness
5	The system shall execute confirmed transactions on Sepolia and return a tx hash with success receipt.	**Requirement**: demo scope
6	The system shall estimate gas/fees and abort if the total exceeds a user-specified max.	**Requirement**: measurable on every run
7	The system shall log the pipeline states (parse → route → simulate → sign → send) with a run ID, without persisting private keys.	**Requirement**: verifiable by log inspection
8	The system shall handle failure modes (RPC error, insufficient funds, bad slippage, rate-limit) with	**Requirement**: fault-injection testable

	actionable error messages.	
9	Signing shall occur client-side or with a limited-fund test account; the system shall not take custody of user private keys.	**Constraint**: external limitation on design
10	The agent shall not autonomously decide *when* to trade; it executes only upon user instruction.	**Constraint**: per project definition
11	The implementation shall leverage or fork Spartan where practical for execution adapters.	**Constraint**: project premise
12	Median time from user confirmation to mined receipt shall be ≤ 12 s (p95 ≤ 25 s) on Sepolia under normal RPC conditions.	**Objective**: latency
13	Executed swap output shall remain within the user-specified slippage tolerance in ≥ 95% of successful swaps.	**Objective**: execution quality

Appendix B: Gantt Chart



Appendix C: Financial Plan

The following table outlines the necessary expenditures and available funding of our project, which revolves primarily around api usage and developer time.

Table 3: Project Budget Table

Consumables/Services					
Item	**Priority**	**Cost/unit**	**Quantity (# of Hours)**	**Total Cost**	**Requires Funding**
LLM API Usage (OpenAI/Anthropic)	1	$25/1 million tokens	2 million tokens	$50	Y
Internet Data Plan	1	$100/month	4 months	$400	N
Github	1	free	N/A	$0	N
TestNet Access	1	free	N/A	$0	N
Cloud database using AWS	1	$0.115 per GB-month	4 months <1 GB	$1	Y
Telegram API Usage	1	free	N/A	$0	N
Domain / UI hosting	2	$15/year	1 year	$15	Y
Total Consumables/Services				$466	
Total Required Funding				$66	
Capital Equipment (None required for this project)					
Item	**Priority**	**Cost/unit**	**Quantity (# of Hours)**	**Total Cost**	**Requires Funding**
Total Capital Equipment				$0	
Total Required Funding				$0	
Student Labor					
Item	**Cost/unit**		**Quantity (# of Hours)**	**Total Cost**	
Ozod Musaev	$30/hour		100	$3,000	
Sami Najim	$30/hour		100	$3,000	
Youssef Fikry	$30/hour		100	$3,000	
George Li	$30/hour		100	$3,000	
Total Student Labor				$12,000	

(Unfunded)			
Project Totals			
Total Cost of Project			$12,466
Total Cost of Required Funding			$66
Available Funding			
Source of Funding		Available amount	
Students ($100 each)		$400	
Supervisor		$100	
Requested from ECE496		$0	
Other		$0	
Total		$500	

Contingency Plan
Our available funding of $500 is well above our required funding of $66 for access to cloud databases and LLM API tokens. This margin provides a substantial buffer against any unexpected cost increases or funding delays and reductions. However, for completeness, we outline contingency measures for extreme scenarios where costs might exceed expectations or funding is temporarily unavailable.

Potential Budget Impact	**Contingency Plan**
Unexpected rises in API usage or cloud hosting fees could increase required funding	Reduce model size, switch to free-tier services, compress or archive old logs
Delays in supervisor funding	Temporarily cover costs with personal contributions and request for reimbursements once funding is available
Funding Reduction	Focus on high priority items including LLM tokens and database access. If necessary, leverage free tier services, and switch to smaller models